

Socratic Texting:

Summary of Your Requirements

Requirement	Selected Option
Application Type	Hybrid Web Application
Execution	Sequential & Parallel
Test Data	Multiple Data Sources (Excel, JSON, DB, API)
Design Pattern	POM + Component-Based Design
Reporting	Multiple Reporting Formats
Environment Support	Config File + CLI + CI/CD
Browser Support	Local + Grid + Cloud
Test Execution	XML + Groups + Runtime Filters
Synchronization	Selenium Explicit Waits
Retry Strategy	Retry Transient Failures Only
CI/CD	Multiple Platforms
Extensibility	Enterprise Automation Platform
Locator Strategy	To Be Decided Later
Logging	Detailed Logging
Utilities	Shared Utility Classes
Audience	Open Source / Reusable

Solution Architecture

Based on your answers, I recommend a **Layered Enterprise Automation Architecture**.

Test Layer

|

TestNG Test Classes



Business Layer

(Reusable Workflows)



Page Object Layer



Component Object Layer



Selenium Wrapper Layer



WebDriver Management



Browser/Grid/Cloud

Supporting Layers

Configuration

Data Providers

Reports

Logging

Utilities

Retry

Listeners

Screenshots

Test Data

This architecture keeps the framework:

- highly maintainable

- scalable
- reusable
- loosely coupled

Why this architecture?

Instead of

Test

↓

Driver

↓

Selenium

we build

Test

↓

Business Flow

↓

Pages

↓

Components

↓

Driver

↓

Browser

This separates responsibilities.

Design Decisions

1. Maven

Why

Dependency management

Build management

Profiles

CI Friendly

Alternative

Gradle

2. TestNG

Why

Parallel Execution

Listeners

Retry

Groups

XML

Alternative

JUnit 5

3. POM + Component Model

Example

Login Page

contains

Header Component

Menu Component

Footer Component

Advantages

Less duplicate code

Easy maintenance

Reusable UI

Alternative

Screenplay Pattern

4. Data Driven

Supports

Excel

JSON

Properties

API

Database

without changing Test Classes.

5. Browser Factory

Instead of

```
new ChromeDriver();
```

we use

```
DriverFactory.getDriver();
```

Later it can create

Chrome

Edge

Firefox

Grid

BrowserStack

LambdaTest

without modifying tests.

6. Configuration Manager

Instead of

```
driver.get("https://qa");
```

use

```
Config.getBaseUrl();
```

Now only properties file changes.

7. Listener

Automatically

Screenshot

Logging

Report Update

Failure Capture

Retry

No code inside tests.

8. Utility Layer

Contains

Excel Utility

JSON Utility

Screenshot Utility

Date Utility

Database Utility

API Utility

Encryption Utility

Zip Utility

Recommended Project Structure

AutomationFramework

|

|— src

| └─ main

| |

| └─ java

| |

| | └─ base

| | | BaseTest.java

| | | BasePage.java

| | |

| | └─ driver

| | | DriverFactory.java

| | | DriverManager.java

| | |

| | └─ pages

| | | LoginPage.java

| | | HomePage.java

| | |

| | | └─ components

| | | Header.java

| | | Menu.java

| | |

| | | └─ listeners

| | | TestListener.java

| | |

| | | └─ retry

| | | RetryAnalyzer.java

| | |

| | └─ reports

| | | ExtentManager.java

| | | AllureManager.java

| | |

| | └─ config

| | | ConfigManager.java

| | |

| | └─ data

| | | ExcelReader.java

| | | JsonReader.java

| | | DBReader.java

| | |

| | └─ utils

| | | WaitUtil.java

| | | ScreenshotUtil.java

| | | LoggerUtil.java

| | |

| | └─ constants

| | FrameworkConstants.java

|

└─ test

| └─ java

| | LoginTest.java

| | SearchTest.java

| |

| └─ resources

| testdata

| config

|

|— reports

|— screenshots

|— logs

|— pom.xml

└─ testng.xml

Industry Best Practices

Driver

✓ ThreadLocal Driver

✓ No Static Driver

✓ Factory Pattern

Wait

✓ Explicit Wait

✓ No Thread.sleep()

Test

- ✓ One Assertion Per Business Validation
- ✓ Independent Tests
- ✓ No Hardcoded Data

Data

- ✓ Externalize Everything
- ✓ No Test Data Inside Java

Reporting

- ✓ Screenshot on Failure
- ✓ Log Every Step
- ✓ Capture Browser Information

CI

- ✓ Maven Commands
- ✓ Environment Parameter
- ✓ Browser Parameter
- ✓ Headless Support

Code

- ✓ Single Responsibility Principle
- ✓ Open Closed Principle
- ✓ DRY

✓ SOLID

Framework Roadmap

To keep the implementation structured and production-ready, I recommend building the framework in the following order:

1. **Project setup** (Maven, dependencies, package structure)
2. **Configuration management** (properties, environment selection)
3. **Driver management** (DriverFactory, ThreadLocal, browser selection)
4. **Base classes** (BaseTest, BasePage)
5. **Page Object & Component layers**
6. **Wait and utility classes**
7. **Data providers** (Excel, JSON, Database, API)
8. **Reporting** (Extent Reports, Allure)
9. **Logging** (Log4j2/SLF4J)
10. **Retry mechanism and TestNG listeners**
11. **Parallel execution support**
12. **CI/CD integration**
13. **Grid and cloud execution**
14. **Sample end-to-end test suite**
15. **Documentation and onboarding guide**